Crash Course in R

Paul Kieffaber

09/02/2025

Welcome!

This document serves two purposes: 1. To demonstrate the text formatting capabilities of R Markdown. 2. To provide a hands-on introduction to the R programming language itself.

1. R Markdown Formatting Basics

R Markdown allows you to create beautifully formatted documents by writing in plain text. It uses a simple syntax to create headers, lists, links, and more. The real power comes from combining this formatted text with live R code chunks.

Headers

You can create section headers using the `#` symbol. More `#` symbols create smaller sub-headers.

```
# Header 1
## Header 2
### Header 3
```

Text Emphasis

You can make text *italic* or **bold**.

- *Italic text* is created with `*asterisks*` or `_underscores_`.
- **Bold text** is created with `**double asterisks**` or `__double underscores__`.
- You can even do ***bold and italic*** with `***triple asterisks***`.

Lists

You can create bulleted (unordered) and numbered (ordered) lists.

Unordered List:

```
* Item A
* Item B
  * Sub-item B1
  * Sub-item B2
```

Ordered List:

```
1. First item
2. Second item
3. Third item
```

Links and Images

- Create a hyperlink with `[display text](URL)`. For example: The R Project.

- Embed an image with `![alt text](path/to/image.png)`.

Blockquotes and Inline Code

> To create a blockquote, start a line with `>`. This is useful for highlighting quotes or important notes.

Sometimes you want to mention a `variable_name` or `function()` in your sentence. You can format it as code by wrapping it in backticks: `` `code` ``.

R Code Chunks

As you'll see throughout this document, R code is placed in "chunks" that look like this...

"'{r} # R code goes here 2 + 2 "'

The output of the code will appear directly below the chunk in the final, knitted document.

---

2. Installing and Loading Packages

R's power comes from its packages, which are collections of functions and data that extend R's capabilities. You only need to **install** a package once, but you need to **load** it every time you start a new R session.

Installing a Package

To install a package, you use the `install.packages()` function. We will install `dplyr`, a very popular package for data manipulation. The line is "commented out" with a `#` so it doesn't run every time we knit the document. You would run this line directly in your R console to install it.

```
# Remove the '#' and run this line in your console to install the package
# install.packages("dplyr")
```

Loading a Package

Once a package is installed, you load it into your R session with the `library()` function.

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##     filter, lag
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

---

3. R as a Calculator: Mathematical Operators

At its simplest, R can be used as a powerful calculator.

```
# Addition
5 + 3
```

```
## [1] 8
```

```r
# Subtraction
10 - 4
```

```
## [1] 6
```

```r
# Multiplication
6 * 7
```

```
## [1] 42
```

```r
# Division
20 / 4
```

```
## [1] 5
```

```r
# Exponentiation (raising to a power)
2^3  # 2 cubed
```

```
## [1] 8
```

```r
# Modulus (finds the remainder of a division)
10 %% 3 # Remainder of 10 divided by 3 is 1
```

```
## [1] 1
```

---

4. Relational and Logical Operators

These operators are used to compare values and result in a `TRUE` or `FALSE` answer. They are the foundation of making decisions in your code.

Relational Operators

- `==` : Equal to
- `!=` : Not equal to
- `<` : Less than
- `>` : Greater than
- `<=` : Less than or equal to
- `>=` : Greater than or equal to

```r
x <- 10
y <- 5

# Is x equal to 10?
x == 10
```

```
## [1] TRUE
```

```r
# Is y not equal to 5?
y != 5
```

```
## [1] FALSE
```

```r
# Is x greater than y?
x > y
```

```
## [1] TRUE
```

Logical Operators

- `&` : AND (both conditions must be true)
- `|` : OR (at least one condition must be true)
- `!` : NOT (reverses the logical state)

```
# Is x greater than 5 AND y less than 10?
(x > 5) & (y < 10)
```

```
## [1] TRUE
```

```
# Is x equal to 5 OR y equal to 5?
(x == 5) | (y == 5)
```

```
## [1] TRUE
```

```
# Is x NOT greater than y?
!(x > y)
```

```
## [1] FALSE
```

---

5. Scripts and Functions

While you can type commands directly into the console, it's better to write them in a script (`.R` file) or an R Markdown file (`.Rmd`) to save your work.

A **function** is a reusable block of code that performs a specific task. You can write your own functions to make your code more organized and efficient.

Defining a Function

Here, we'll create a function to convert temperature from Celsius to Fahrenheit.

```
celsius_to_fahrenheit <- function(celsius_temp) {
  # The formula is (Celsius * 9/5) + 32
  fahrenheit_temp <- (celsius_temp * 9/5) + 32
  return(fahrenheit_temp)
}
```

Using a Function

Now that we've defined the function, we can "call" it with different inputs.

```
# Freezing point
freezing_F <- celsius_to_fahrenheit(0)
print(paste("0°C is", freezing_F, "°F"))
```

```
## [1] "0°C is 32 °F"
```

```
# Boiling point
boiling_F <- celsius_to_fahrenheit(100)
print(paste("100°C is", boiling_F, "°F"))
```

```
## [1] "100°C is 212 °F"
```

Note: While both c() and paste() are used to combine things, they do so for different purposes and produce different results.

Here's the simplest way to think about it:

`c()` creates a vector of data. Think of it as putting items into a container or a list.

`paste()` creates character strings for display. Think of it as writing a sentence by joining words together.

---

6. Variables and Data Types

In R, you can store values in **variables** so you can use them later. The assignment operator is `<-`.

Common Data Types

R has several fundamental data types.

```
# Numeric: For numbers (integers and decimals)
my_number <- 42
another_number <- 3.14

# Character: For text (strings), always in quotes
my_text <- "Hello, R!"

# Logical: For TRUE or FALSE values
is_learning <- TRUE

# We can print the variables to see their values
print(my_number)
```

```
## [1] 42
```

```
print(my_text)
```

```
## [1] "Hello, R!"
```

```
print(is_learning)
```

```
## [1] TRUE
```

Vectors

A **vector** is a one-dimensional collection of elements of the *same* data type. You can create a vector using the `c()` function (which stands for 'combine').

```
# A numeric vector
numeric_vector <- c(1, 10, 50, 100)

# A character vector
character_vector <- c("apple", "banana", "cherry")

print(numeric_vector)
```

```
## [1]   1  10  50 100
```

```
print(character_vector)
```

```
## [1] "apple"  "banana" "cherry"
```

Data Frames

The most common way to store tabular data in R is in a **data frame**. Think of it as a spreadsheet, with rows representing observations and columns representing variables.

Creating a Data Frame

```
# Create a few vectors
student_id <- c(1, 2, 3, 4)
student_name <- c("Alice", "Bob", "Charlie", "David")
test_score <- c(88, 92, 75, 85)

# Combine them into a data frame
student_data <- data.frame(
  ID = student_id,
  Name = student_name,
  Score = test_score
)

# Print the data frame
print(student_data)
```

```
##   ID    Name Score
## 1  1   Alice    88
## 2  2     Bob    92
## 3  3 Charlie    75
## 4  4   David    85
```

Inspecting a Data Frame

There are several useful functions to get a quick overview of your data.

```
# head() shows the first few rows
head(student_data)
```

```
##   ID    Name Score
## 1  1   Alice    88
## 2  2     Bob    92
## 3  3 Charlie    75
## 4  4   David    85
```

```
# tail() shows the last few rows
tail(student_data)
```

```
##   ID    Name Score
## 1  1   Alice    88
## 2  2     Bob    92
## 3  3 Charlie    75
## 4  4   David    85
```

```
# str() shows the structure of the data frame (str-ucture)
str(student_data)
```

```
## 'data.frame':    4 obs. of  3 variables:
##  $ ID   : num  1 2 3 4
##  $ Name : chr  "Alice" "Bob" "Charlie" "David"
##  $ Score: num  88 92 75 85
```

```
# summary() provides summary statistics for each column
summary(student_data)
```

```
##        ID           Name               Score
##  Min.   :1.00   Length:4           Min.   :75.0
```

```
##  1st Qu.:1.75   Class :character   1st Qu.:82.5
##  Median :2.50   Mode  :character   Median :86.5
##  Mean   :2.50                      Mean   :85.0
##  3rd Qu.:3.25                      3rd Qu.:89.0
##  Max.   :4.00                      Max.   :92.0
```

Indexing and Slicing: Accessing Your Data

Once you have data stored in a vector or a data frame, you need a way to access specific parts of it. This process is called **indexing** (accessing elements) or **slicing** (accessing a sequence of elements). R is a **1-indexed** language, which means the first element is at position 1, the second at position 2, and so on.

Indexing Vectors

We use square brackets `[]` to access elements of a vector.

```
# Let's create a vector to work with
ages <- c(25, 31, 19, 42, 50, 22)
```

1. Accessing a Single Element by Position To get the element at a specific position, place the position number in the brackets.

```
# Get the third element
ages[3]
```

```
## [1] 19
```

2. Accessing Multiple Elements by Position You can pass a vector of positions into the brackets to get multiple elements.

```
# Get the first and fifth elements
ages[c(1, 5)]
```

```
## [1] 25 50
```

3. Accessing a Sequence of Elements Use the colon operator `:` to get a continuous slice of the vector.

```
# Get all elements from the second to the fourth
ages[2:4]
```

```
## [1] 31 19 42
```

```
# Get all elements from the first to the last, by two.
ages[seq(1,length(ages),by=2)]
```

```
## [1] 25 19 50
```

4. Logical Indexing This is a very powerful technique. You can use a logical vector (`TRUE`/`FALSE` values) to select only the elements corresponding to a `TRUE` value. This is most often used for filtering.

```
# Which ages are over 30?
ages > 30
```

```
## [1] FALSE  TRUE FALSE  TRUE  TRUE FALSE
```

```
# Now use that logical vector to select ONLY the ages over 30
ages[ages > 30]
```

```
## [1] 31 42 50
```

5. Excluding Elements Use a negative number to exclude the element at that position.

```r
# Get all ages EXCEPT the second one
ages[-2]
```

```
## [1] 25 19 42 50 22
```

Indexing Data Frames

Data frames have two dimensions: rows and columns. We still use square brackets `[]`, but the notation is `[rows, columns]`. The comma is crucial!

```r
# Let's create a data frame to work with
student_data <- data.frame(
  ID = c(101, 102, 103, 104),
  Name = c("Alice", "Bob", "Charlie", "David"),
  Score = c(88, 92, 75, 85),
  Major = c("Biology", "History", "Math", "Biology")
)
print(student_data)
```

```
##    ID    Name Score   Major
## 1 101   Alice    88 Biology
## 2 102     Bob    92 History
## 3 103 Charlie    75    Math
## 4 104   David    85 Biology
```

1. The `$` Operator (Most Common) The easiest and most common way to select an entire column by its name is with the `$` operator.

```r
# Get the entire 'Score' column
student_data$Score
```

```
## [1] 88 92 75 85
```

```r
# Calculate the mean of the 'Score' column
mean(student_data$Score)
```

```
## [1] 85
```

2. Using `[rows, columns]` Notation

- To select **rows**, specify the row number(s) *before* the comma.
- To select **columns**, specify the column number(s) or name(s) *after* the comma.
- Leave one side blank to select *all* rows or *all* columns.

```r
# Get the entire third row
student_data[3, ]
```

```
##    ID    Name Score Major
## 3 103 Charlie    75  Math
```

```r
# Get the first and third rows
student_data[c(1, 3), ]
```

```
##    ID    Name Score   Major
## 1 101   Alice    88 Biology
## 3 103 Charlie    75    Math
```

```r
# Get the entire 'Name' column (the 2nd column)
# Note: This returns a vector
student_data[, 2]
```

```
## [1] "Alice"   "Bob"     "Charlie" "David"
```

```
# Get a specific cell: the name of the student in the 3rd row
student_data[3, 2]
```

```
## [1] "Charlie"
```

```
# Get multiple columns by name
student_data[, c("Name", "Major")]
```

```
##      Name   Major
## 1   Alice Biology
## 2     Bob History
## 3 Charlie    Math
## 4   David Biology
```

3. Filtering Data Frames (Very Important!) This combines the logical indexing we saw with vectors and the `[rows, columns]` notation. We create a logical condition to select which rows we want to keep.

```
# Create a logical vector: Which students scored above 85?
student_data$Score > 85
```

```
## [1]  TRUE  TRUE FALSE FALSE
```

```
# Now, use that logical vector in the 'rows' position to filter the data frame
high_scorers <- student_data[student_data$Score > 85, ]
print(high_scorers)
```

```
##    ID  Name Score   Major
## 1 101 Alice    88 Biology
## 2 102   Bob    92 History
```

```
# You can also filter by text
# Find all the Biology majors
biology_majors <- student_data[student_data$Major == "Biology", ]
print(biology_majors)
```

```
##    ID  Name Score   Major
## 1 101 Alice    88 Biology
## 4 104 David    85 Biology
```

---

7. Flow Control and Loops

Flow control allows you to direct the "flow" of your script, making decisions and repeating tasks automatically. This is where you move from just running commands to writing intelligent programs.

Conditional Statements (`if`, `else if`, `else`)

Conditional statements let you run specific chunks of code only if certain conditions are met.

The `if` Statement The simplest conditional is an `if` statement. The code inside the curly braces `{}` will only run if the condition in the parentheses `()` is `TRUE`.

```
x <- 15

if (x > 10) {
  print("The variable x is greater than 10.")
}
```

```
## [1] "The variable x is greater than 10."
```

Since `x` is 15 (which is greater than 10), the message was printed. If `x` was 5, nothing would have happened.

Adding `else` You can add an `else` statement to provide an alternative block of code to run if the `if` condition is `FALSE`.

```
temperature <- 12 # in Celsius

if (temperature > 25) {
  print("It's a hot day!")
} else {
  print("It's not a hot day.")
}
```

```
## [1] "It's not a hot day."
```

Chaining Conditions with `else if` You can check multiple conditions in a sequence using `else if`. R will check them one by one and run the code for the *first* one that is `TRUE`. If none are true, the final `else` block is run.

```
grade <- 85

if (grade >= 90) {
  print("You got an A!")
} else if (grade >= 80) {
  print("You got a B.")
} else if (grade >= 70) {
  print("You got a C.")
} else {
  print("You need to study more.")
}
```

```
## [1] "You got a B."
```

Loops (`for` and `while`)

Loops are used to repeat a block of code multiple times without having to write it out over and over.

The `for` Loop A `for` loop is perfect for when you want to iterate over every item in a sequence (like a vector).

Example 1: Looping over numbers This loop will go through the `numbers` vector, and in each iteration, the current item will be assigned to the variable `num`.

```
numbers <- c(1, 2, 3, 4, 5)

for (num in numbers) {
  squared_num <- num^2
  print(paste("The square of", num, "is", squared_num))
}
```

```
## [1] "The square of 1 is 1"
## [1] "The square of 2 is 4"
## [1] "The square of 3 is 9"
## [1] "The square of 4 is 16"
## [1] "The square of 5 is 25"
```

Example 2: Looping over text The same logic applies to character vectors.

```
fruits <- c("apple", "banana", "cherry")

for (fruit in fruits) {
  print(paste("I would like to eat a", fruit))
}
```

```
## [1] "I would like to eat a apple"
## [1] "I would like to eat a banana"
## [1] "I would like to eat a cherry"
```

The `while` Loop A `while` loop will continue to run as long as its condition is `TRUE`. You have to make sure that something inside the loop changes so that the condition eventually becomes `FALSE`, otherwise you'll create an infinite loop!

This `while` loop implements a simple countdown.

```
countdown <- 5

while (countdown > 0) {
  print(countdown)
  # IMPORTANT: We decrease the value of countdown in each iteration.
  # This ensures the loop will eventually stop.
  countdown <- countdown - 1
}
```

```
## [1] 5
## [1] 4
## [1] 3
## [1] 2
## [1] 1
```

```
print("Blast off!")
```

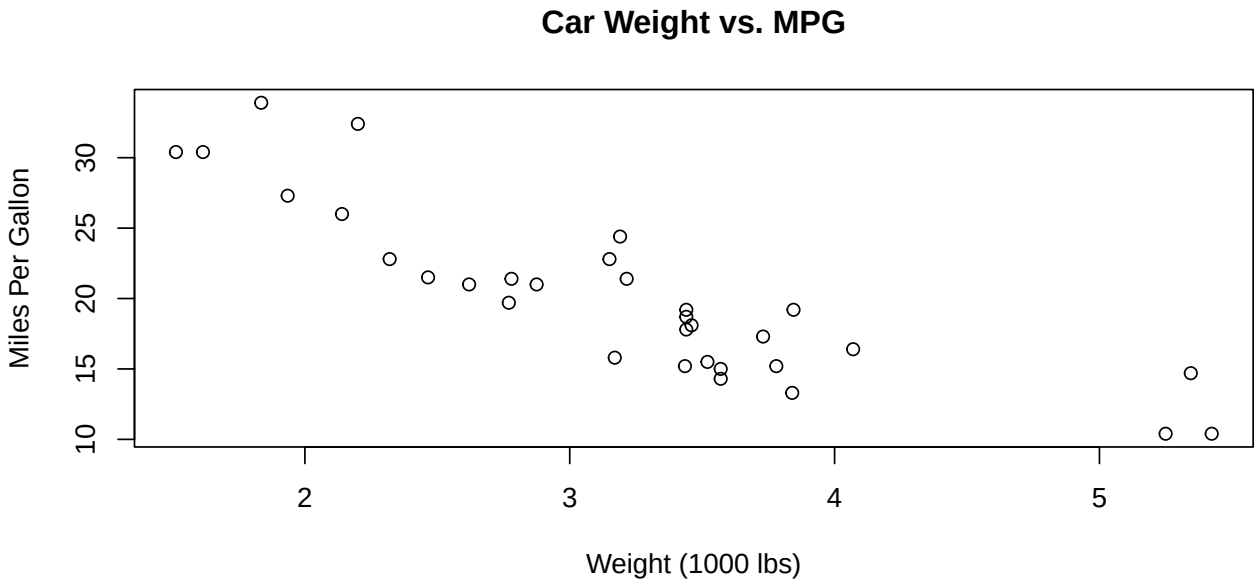
```
## [1] "Blast off!"
```

---

8. Basic Plotting

R is fantastic for data visualization. Here's a quick look at creating a simple plot using R's built-in plotting functions. We'll use the built-in `mtcars` dataset.

```
# The plot() function is great for simple scatterplots
# We'll plot car weight (wt) vs. miles per gallon (mpg)
plot(
  x = mtcars$wt,
  y = mtcars$mpg,
  main = "Car Weight vs. MPG",
  xlab = "Weight (1000 lbs)",
  ylab = "Miles Per Gallon"
)
```



This is just the beginning. Packages like `ggplot2` provide a powerful and flexible system for creating a huge variety of beautiful plots. This document has covered some of the fundamental building blocks to get you started on your journey with R!

Assignment: Practice Problems

Now it's your turn to apply what you've learned. Write the R code to solve each problem in the empty code chunks provided below. Be sure to annotate each line using the comment `#` character to eplain what each lin of your code does.

Part 1: Variables and Basic Operations

Problem 1: Area of a Rectangle

Create two variables, `width` and `height`, and assign them numeric values (e.g., 10 and 5). Calculate the area of the rectangle (`width * height`) and store it in a variable called `area`. Print the `area`.

```
# Your code here
width <- 10
height <- 5
area= width * height
print(paste("the area is",area))
```

```
## [1] "the area is 50"
```

Problem 2: Full Name Greeting

Create a variable `first_name` and a variable `last_name`. Use the `paste()` function to combine them into a single string that says "Hello, [First Name] [Last Name]!" and print the result.

```
# Your code here
first_name <- "Kat"
last_name <- "Gour!"
print(paste("Hello,",first_name,last_name))
```

```
## [1] "Hello, Kat Gour!"
```

Problem 3: Is the Number Even?

Write a **function** that takes a number as input and determines whether or not that number is odd or even, returning the logical `TRUE` if the number is even or `FALSE` if the number is odd. (Hint: the modulus operator `%%` might come in handy here). Call the function twice after you've created it to demonstrate that it works.

```
# Function to check if a number is odd or even
check_odd_even <- function(number) {
  if (number %% 2 == 0) {
    return("even")
  } else {
    return("odd")
  }
}

# Test 2x
number_to_check <- 12
result <- check_odd_even(number_to_check)
cat(number_to_check, "is an", result, "number. ")
```

```
## 12 is an even number.
```

```
number_to_check <- 15
result <- check_odd_even(number_to_check)
cat(number_to_check, "is an", result, "number.")
```

```
## 15 is an odd number.
```

Part 2: Vectors and Logical Operators

Problem 4: Monthly Expenses

Create a numeric vector named `expenses` that contains the following values: `800 250 120 300 150`. These represent your monthly expenses for rent, groceries, utilities, transportation, and entertainment. Calculate the total of all expenses using the `sum()` function.

```
# Your code here
expenses <- c(800, 250, 120, 300, 150)
sum(expenses)
```

```
## [1] 1620
```

Problem 5: Accessing Vector Elements

Using the `expenses` vector from the previous problem, write code to access and print only the expense for groceries (the 2nd element) and the expense for entertainment (the 5th element).

```
# Your code here
expenses[c(2, 5)]
```

```
## [1] 250 150
```

Problem 6: Which Expenses are High?

Using the `expenses` vector, write a line of code that returns a logical vector (`TRUE`/`FALSE` values) indicating which expenses are greater than `200`.

```
expenses <- c(800, 250, 120, 300, 150)
expenses >= 200
```

```
## [1]  TRUE  TRUE FALSE  TRUE FALSE
```

```
expenses[expenses >= 200]
```

```
## [1] 800 250 300
```

Problem 7: Checking Expenses in a Data Frame

Create a data frame with the following column names: (1) Month (2) Rent, (3) Groceries, (4) Travel, and (5) Holiday. The values in the month column should include December-May Use any reasonable $ values for Rent, Groceries, and Travel each of the six months. The Holiday column should contain logical values (i.e., TRUE or FALSE) indicating whether or not the month contains a Holiday (this may vary from person to person).

Assuming that travel costs are likely to be higher in months where there is a holiday, use indexing and the `mean()` function to compute the average travel expense in months where there is a holiday and in months that do not include a holiday.

```
# Your code here
# Create a few vectors
Month <- c("December", "January", "February", "March", "April", "May")
Rent <- c(900, 950, 950, 950, 950, 950)
Groceries <- c(150, 100, 120, 90, 105, 100)
Travel <- c(50, 20, 30, 20, 50, 35)
Holiday <- c("TRUE","TRUE","TRUE","FALSE","FALSE","FALSE")
# Combine them into a data frame
yearly_data <- data.frame(
Months = Month,
Rents = Rent,
Shopping = Groceries,
Travels = Travel,
Holidays = Holiday
)
# Print the data frame
print(yearly_data)
```

```
##     Months Rents Shopping Travels Holidays
## 1 December   900      150      50     TRUE
## 2  January   950      100      20     TRUE
## 3 February   950      120      30     TRUE
## 4    March   950       90      20    FALSE
## 5    April   950      105      50    FALSE
## 6      May   950      100      35    FALSE
```

Part 3: Flow Control and Loops

Problem 8: Movie Rating Guide Imagine a movie rating system. Write an `if/else if/else` statement that checks a variable `rating` and prints: - "Excellent!" if the rating is 8 or higher. - "Good" if the rating is between 5 and 8 (exclusive of 8). - "Poor" for anything else. Test your code with `rating <- 8`, `rating<-6`, and `rating<-2`.

```
rating <- 4
# Your if/else if/else statement here
if (rating >= 8) {
print("Excellent!")
} else if (rating >= 5) {
print("Good.")
} else {
```

```
print("Poor.")
}
```

```
## [1] "Poor."
```

Problem 9: Temperature Conversion Loop You have a vector of temperatures in Celsius. Use a `for` loop to iterate through each temperature, convert it to Fahrenheit, and print the result. The formula is `(Celsius * 9/5) + 32`.

```
celsius_temps <- c(0, 15, 30, -5)

# Your for loop here
for (num in celsius_temps) {
farenheit <- ((num*(9/5)+32))
print(paste("The celsius temparature", num, "is", farenheit, "in farenheit"))
}
```

```
## [1] "The celsius temparature 0 is 32 in farenheit"
## [1] "The celsius temparature 15 is 59 in farenheit"
## [1] "The celsius temparature 30 is 86 in farenheit"
## [1] "The celsius temparature -5 is 23 in farenheit"
```

Problem 10: Reaching a Savings Goal You want to save $50. You start with $10 and save $5 each week. Use a `while` loop to determine how many weeks it will take to reach your goal. Inside the loop, you should increment the weeks and add to your savings. Print the final number of weeks.

```
savings <- 10
goal <- 50
weeks <- 0

# Your while loop here
while (savings < goal) {
  savings <- savings + 5
  weeks <- weeks + 1
}

print(paste("It will take", weeks, "weeks to reach the goal."))
```

```
## [1] "It will take 8 weeks to reach the goal."
```

Part 4: Data Frames and Plotting with `mtcars`

Problem 11: Create a Subset Data Frame The `mtcars` dataset is already built into R. For this first step, create a *new* data frame called `my_cars` that contains only three columns from `mtcars`: `mpg` (Miles Per Gallon), `hp` (Horsepower), `wt` (Weight), and `cyl` (Cylinders). (Hint: `new_df <- old_df[, c("col1", "col2")]`). Print the first few rows of your new data frame using `head()`.

```
# Your code here
my_cars <- mtcars[, c('mpg','hp','wt','cyl')]
head(my_cars)
```

```
##                   mpg  hp    wt cyl
## Mazda RX4         21.0 110 2.620   6
## Mazda RX4 Wag     21.0 110 2.875   6
## Datsun 710        22.8  93 2.320   4
## Hornet 4 Drive    21.4 110 3.215   6
```

```
## Hornet Sportabout 18.7 175 3.440   8
## Valiant           18.1 105 3.460   6
```

Problem 12: Inspect Your New Data Frame Using the `my_cars` data frame you just created, use the functions `str()` and `summary()` to inspect its structure and get summary statistics for the three columns.

```
# Your code here
str(my_cars)
```

```
## 'data.frame':    32 obs. of  4 variables:
##  $ mpg: num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ hp : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ wt : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ cyl: num  6 6 4 6 8 6 8 4 4 6 ...
```

```
summary(my_cars)
```

```
##       mpg              hp              wt             cyl
##  Min.   :10.40   Min.   : 52.0   Min.   :1.513   Min.   :4.000
##  1st Qu.:15.43   1st Qu.: 96.5   1st Qu.:2.581   1st Qu.:4.000
##  Median :19.20   Median :123.0   Median :3.325   Median :6.000
##  Mean   :20.09   Mean   :146.7   Mean   :3.217   Mean   :6.188
##  3rd Qu.:22.80   3rd Qu.:180.0   3rd Qu.:3.610   3rd Qu.:8.000
##  Max.   :33.90   Max.   :335.0   Max.   :5.424   Max.   :8.000
```

Problem 13: Calculate Average Horsepower Using the `$` operator to access the `hp` column of your `my_cars` data frame, calculate the average horsepower separately for the 4-cylinder and 8-cylinder cars in your dataset using the `mean()` function.

```
# Your code here
mean(my_cars$hp[my_cars$cyl == 4])
```

```
## [1] 82.63636
```

```
mean(my_cars$hp[my_cars$cyl == 8])
```

```
## [1] 209.2143
```

Problem 14: Find the High-Power Cars Write code to display the *rows* from the original `mtcars` data frame for only the cars with a horsepower (`hp`) greater than 200.

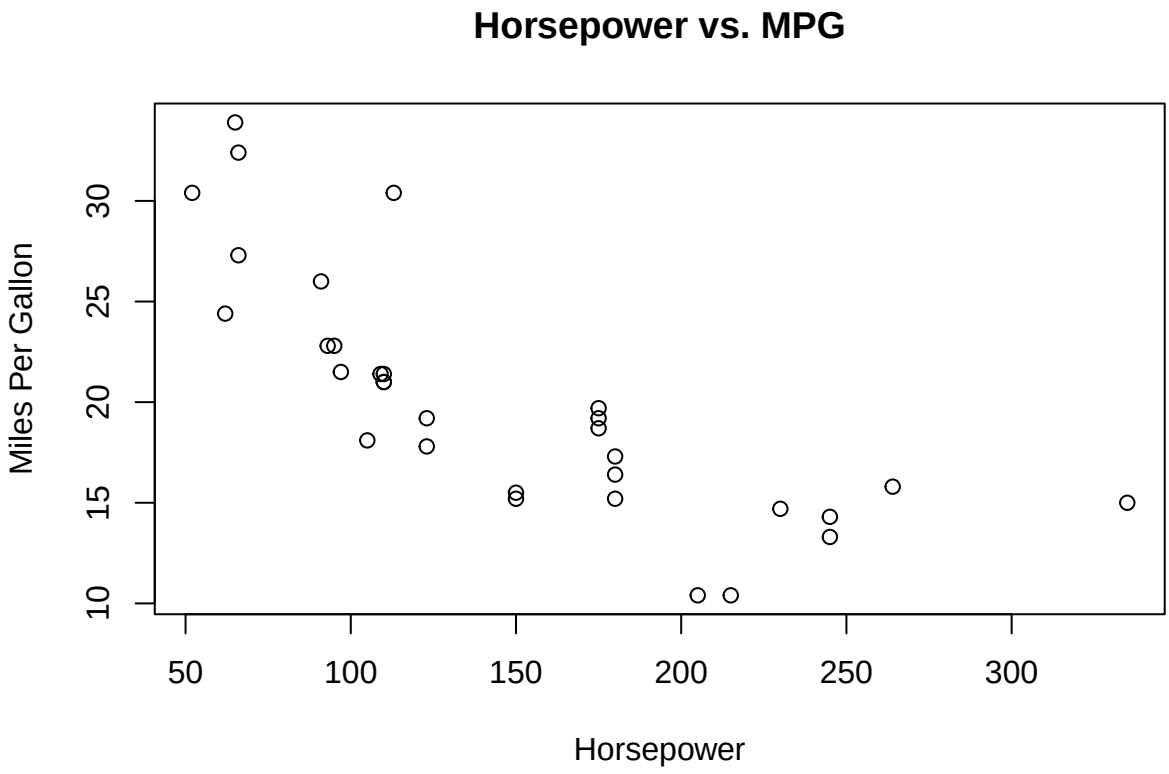
```
# Your code here
mtcars[mtcars$hp > 200, ]
```

```
##                     mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Duster 360         14.3   8  360 245 3.21 3.570 15.84  0  0    3    4
## Cadillac Fleetwood 10.4   8  472 205 2.93 5.250 17.98  0  0    3    4
## Lincoln Continental 10.4  8  460 215 3.00 5.424 17.82  0  0    3    4
## Chrysler Imperial  14.7   8  440 230 3.23 5.345 17.42  0  0    3    4
## Camaro Z28         13.3   8  350 245 3.73 3.840 15.41  0  0    3    4
## Ford Pantera L     15.8   8  351 264 4.22 3.170 14.50  0  1    5    4
## Maserati Bora      15.0   8  301 335 3.54 3.570 14.60  0  1    5    8
```

Problem 15: Plot hp vs. mpg Using the full `mtcars` dataset, create a scatterplot to visualize the relationship between car horse power (`hp`) and fuel efficiency (`mpg`). Put `wt` on the x-axis and `mpg` on the y-axis. Use the `plot()` function.

```
# Your code here (plot mtcars data)
plot(
x = mtcars$hp,
```

```
y = mtcars$mpg,
main = "Horsepower vs. MPG",
xlab = "Horsepower",
ylab = "Miles Per Gallon"
)
```



Part 5: Bringing It All Together

Problem 16: Interactive Number Guessing Game This problem will test your ability to combine `readline()`, a `while` loop, and `if/else if/else` statements to create a simple interactive program.

Your Goal: Write a script that plays a number guessing game with the user.

Instructions:

1. **Set a secret number.** Create a variable named `secret_number` and assign it an integer value between 1 and 100 (e.g., `42`).
2. **Create an infinite loop.** Use `while (TRUE)` to create a loop that will run forever until you explicitly tell it to stop.
3. **Get user input.** Inside the loop, use `readline()` to prompt the user to guess a number. Store their input in a variable.
4. **Convert the input.** Remember that `readline()` returns a character string. You must convert the user's input into a number using `as.numeric()` before you can compare it to your `secret_number`.
5. **Check the guess.** Use an `if/else if/else` statement with logical operators to check the user's guess against the `secret_number`:
   - If the guess is **less than** the secret number, print "Too low! Guess again."
   - If the guess is **greater than** the secret number, print "Too high! Guess again."

- If the guess is **equal to** the secret number, print a success message like "Correct! You guessed the number!" and then use the `break` command to exit the `while` loop.
- Keep track of how many guesses the user takes using a `guess_counter` variable that is initially set to 0. Inside the loop, add 1 to the counter for each guess. When the user guesses correctly, include the number of guesses in the success message (e.g., "Correct! You guessed the number in 5 tries!").

```
# 1. Set the secret number
#secret_number <- 42

#print("I'm thinking of a number between 1 and 100.")

# 2. Start the loop{
#guess_counter <- 0

#while (TRUE) {
  #user_input <- readline(prompt = "Enter your guess: ")
  #guess <- as.numeric(user_input)
  #if (is.na(guess)){
  #  print("Please enter a valid number.")
   # next
  #}
  #how many guesses
 # guess_counter <- guess_counter + 1

  #check
 # if (guess < secret_number) {
  #  print("Too low! Guess again.")
 # } else if (guess > secret_number) {
 #   print("Too high! Guess again.")
 # } else {
 #   print(paste("Correct! You guessed the number in", guess_counter, "tries!"))
 #   break
 # }
#}
```

Problem 17: The Fuel Efficiency Expert

Objective: Your goal is to analyze the `mtcars` dataset to automatically generate a performance summary. You will write a script that iterates through the different cylinder groups (4, 6, and 8 cylinders), calculates the average fuel efficiency for each group, and then prints a customized evaluation based on that average.

This problem will require you to use a `for` loop, perform logical indexing to filter a data frame, and use `if/else if/else` statements to make decisions.

Instructions:

1. **Create a vector of cylinder groups.** Make a vector named `cylinder_groups` that contains the unique cylinder values you want to analyze: `c(4, 6, 8)`.
2. **Start a `for` loop.** Write a `for` loop that iterates through each value in your `cylinder_groups` vector.
3. **Inside the loop, filter the data.** For each cylinder count in the loop, use **logical indexing** to create a temporary subset of the `mtcars` data frame that contains *only* the cars with that number of cylinders.
4. **Calculate the average MPG.** For the temporary subset you just created, calculate the mean of the `mpg` column. Store this value in a variable (e.g., `avg_mpg`).
5. **Use conditional logic to evaluate efficiency.** Write an `if/else if/else` statement that checks the `avg_mpg` you just calculated:

- If `avg_mpg` is greater than 25, print a message like: `"Cars with [X] cylinders are highly fuel-efficient."`
- If `avg_mpg` is between 18 and 25 (inclusive), print a message like: `"Cars with [X] cylinders have moderate fuel efficiency."`
- Otherwise (if `avg_mpg` is less than 18), print a message like: `"Cars with [X] cylinders are not fuel-efficient."`

6. Make sure to replace `[X]` in your print statements with the actual cylinder count from the current loop iteration. The `paste()` function will be very helpful here.

Starter Code:

```
# 1. Create a vector of cylinder groups to loop through
cylinder_groups <- c(4, 6, 8)

# 2. Start the for loop
for (cyl in cylinder_groups) {

  # 3. Use logical indexing to create a subset for the current cylinder group
  currentgrp <- mtcars[mtcars$cyl == cyl, ]

  # 4. Calculate the average MPG for the subset
  avg_mpg <- mean(currentgrp$mpg)

  # 5. Use if/else if/else to check the average MPG and print a summary
  if (avg_mpg > 25) {
    print(paste("Cars with", cyl, "cylinders are highly fuel-efficient."))
  } else if (avg_mpg >= 18 && avg_mpg <= 25) {
    print(paste("Cars with", cyl, "cylinders have moderate fuel efficiency."))
  } else {
    print(paste("Cars with", cyl, "cylinders are not fuel-efficient."))
  }
}
```

```
## [1] "Cars with 4 cylinders are highly fuel-efficient."
## [1] "Cars with 6 cylinders have moderate fuel efficiency."
## [1] "Cars with 8 cylinders are not fuel-efficient."
```

Expected Output: When you run your completed code, you should see something similar to the following output in your console:

```
[1] "Cars with 4 cylinders are highly fuel-efficient."
[1] "Cars with 6 cylinders have moderate fuel efficiency."
[1] "Cars with 8 cylinders are not fuel-efficient."
```

Problem 18: The Plant Species Analyst

Objective: You are an assistant to a botanist who is studying iris flowers. The botanist has a theory that the species with the largest average petal area is the most resilient. Your task is to analyze the built-in `iris` dataset to identify which of the three species has the largest average petal area.

This problem will require you to get unique values for a column using the `unique()` function, loop through those values using a `for` loop, perform logical indexing, create a new calculated value, and use an `if` statement with "tracker" variables to find the maximum value across all categories.

Instructions:

1. **Get the unique species.** First, create a vector called `species_names` that contains the unique species from the `iris$Species` column. The `unique()` function will be perfect for this.
2. **Initialize "tracker" variables.** Before you start the loop, you need variables to keep track of the winner. Create two variables:
   - `max_avg_petal_area` and set it to `0`.
   - `winning_species` and set it to an empty string `""`.
3. **Start a `for` loop.** Write a `for` loop that iterates through each name in your `species_names` vector.
4. **Inside the loop, filter the data.** For each species, use **logical indexing** to create a temporary subset of the `iris` data frame.
5. **Calculate petal area.** For your temporary subset, calculate the average petal area. You can estimate this as `mean(subset$Petal.Length * subset$Petal.Width)`. Store this value in a variable like `current_avg_area`.
6. **Use an `if` statement to check for a new winner.** Compare the `current_avg_area` to your `max_avg_petal_area` tracker.
   - If the current species' average petal area is greater than the max area found so far, you have a new winner! You must update both of your tracker variables:
     - Set `max_avg_petal_area` to the `current_avg_area`.
     - Set `winning_species` to the name of the current species.
7. **After the loop, print the final result.** Once the loop has finished checking all the species, your tracker variables will hold the final answer. Print a single, formatted summary sentence announcing the winner. The `paste()` function will be helpful.

Starter Code:

```r
# 1. Get the unique species from the iris dataset
species_names <- unique(iris$Species)

# 2. Initialize tracker variables
max_avg_petal_area <- 0
winning_species <- ""

# 3. Start the for loop to iterate through each species
for (species in species_names) {

  # 4. Use logical indexing to create a subset for the current species
  currentspecies <- iris[iris$Species == species, ]

  # 5. Calculate the average petal area for the subset
  current_avg_area <- mean(currentspecies$Petal.Length * currentspecies$Petal.Width)

  # 6. Use an if statement to check if this species is the new winner
  if (current_avg_area > max_avg_petal_area) {
    max_avg_petal_area <- current_avg_area
    winning_species <- species
  }
}

# 7. Print the final result after the loop is finished
print(paste("The winning species is", winning_species,
            "with an average petal area of", round(max_avg_petal_area, 2)))
```

```
## [1] "The winning species is virginica with an average petal area of 11.3"
```

Expected Output: When your code is run correctly, it should produce a single summary line that looks like this:

```
[1] "The winning species is virginica with an average petal area of 10.88"
```

(Note: Your exact number might vary slightly depending on rounding, which is fine.)